CS 412: Spring '26
Introduction To Data Mining

Assignment 2

(Wednesday, March 26 at 11:59 pm)

- The homework is due on **Wednesday, March 26 at 11:59 p.m.** We will be using Gradescope for homework assignments. Please do NOT email a copy of your solution. Contact the TAs if you are having technical difficulties in submitting the assignment.
- Please use Campuswire if you have questions about the homework. Make sure to tag your post appropriately. Also, scroll through previous posts to make sure that your query was not answered previously. In case you are sending us an email regarding this Assignment, start the subject with "CS 412 HW2:" and include **all** TAs and the Instructor.
- Please write your code entirely by yourself.
- Late submission policy: there will be a 24-hour grace period without any grade reduction, i.e., Gradescope will accept late submissions until **Thursday, March 27, at 23:59**. Unfortunately, We will NOT accept late submissions past the grace period (without a reasonable justification).

  Instructions

  - All programming needs to be in Python 3.
  - The homework will be graded using Gradescope. You will be able to submit your code as many times as you want. There are no post deadline tests. The grade appearing on Gradescope right after your submission will be your final grade for this assignment.
  - Two python files named `homework2_q1.py` and `homework2_q2.py` containing starter code. Do NOT change the filename.
  - Do NOT add any additional `import` lines to your code. Required libraries have already been included in the starter code.
  - For submitting on Gradescope, you would need to upload both files, `homework2_q1.py` and `homework2_q2.py`, to the `Assignment 2 Code link` and the PDF report to the `Assignment 2 PDF link`.

Part 1 Code

1. (35 points) The problem will focus on developing your own code for: `Ensemble Methods`. Your code will be evaluated using a standard classification model applied to a multi-class classification dataset.

   Dataset: We will be using the following dataset for the assignment.

   `Digits`: The `Digits` dataset comes prepackaged with `scikit-learn` (sklearn.datasets.load_digits). The dataset has 1797 points, 64 features, and 10 classes corresponding to ten numbers $0, 1, \ldots, 9$. The dataset was (likely) created from the following dataset:
   `https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits`

   Notes: In this assignment, you will implement simplified versions of bagging and boosting for classification. The goal is to understand how ensemble methods combine weak learners to improve predictive performance.

   You may use numpy and `sklearn.tree.DecisionTreeClassifier` only as base learners, but you may not use `sklearn`'s ensemble classes.

   The starter code found in `homework2_q1.py` and features two classes, `RandomForestClassifier` and `AdaBoostClassifier`.

   (a) **Bagging**: Bagging (Bootstrap Aggregating) trains multiple classifiers on bootstrap-resampled versions of the training data and aggregates their predictions via majority vote.
   You will have to develop **code** for the following two functions in `RandomForestClassifier`:

   i. `fit(self, X, y)`, which fits the bagging classifier on the training data (X, y) by training M decision tree classifiers (base learners) on M bootstrap samples of the training data. Use `sklearn.tree.DecisionTreeClassifier`, and make sure to specify correctly `max_depth` and `random_state`.
   Hint: Look at self.tree_max_depth and `self.random_state`. For the random state, ensure that each tree in the ensemble is initialized with a *different* random state.

   ii. `predict(self, X)`, which predicts the class labels for the input data X by aggregating the predictions from all M base learners using majority vote. Break ties by selecting the lowest class label, e.g., if classes 2,3, and 5 are tied, class 2 wins.

   (b) **Boosting**: AdaBoost builds an ensemble of weak classifiers sequentially, reweighting training examples to focus on harder cases.
   Likewise, you will have to develop **code** for the following two functions in `AdaBoostClassifier`:

   i. `fit(self, X, y)`, which fits the AdaBoost classifier on the training data (X, y) by training M decision stumps (one-level decision trees) sequentially. Use `sklearn.tree.DecisionTreeClassifier` with `max_depth=1` as the base learner.

   ii. `predict(self, X)`, which predicts the class labels for the input data X by aggregating the weighted predictions from all M base learners.

   Pseudocode: Algorithm 1, Adaboost.SAMME (Stagewise Additive Modeling using a Multi-class Exponential loss function) (1), is a **multi-class** variant of the **two-class** AdaBoost algorithm(Freund and Shapire, 1997) introduced in Lecture 13. You will use this **multi-class** version for the implementation of (b).

Algorithm 1 AdaBoost for Multiclass Classification (SAMME)

Require: Training set $D = \{(x_n, y_n)\}_{n=1}^{N}$, classes $y_n \in \{1, \ldots, K\}$, rounds T

Initialize weights $w_n^{(1)} \leftarrow \frac{1}{N}$ for all n
for $t = 1$ to T **do**
  Fit base classifier $h_t : \mathcal{X} \to \{1, \ldots, K\}$ using weights $\{w_n^{(t)}\}$
  Weighted error:

$$\epsilon_t \;\leftarrow\; \frac{\sum_{n=1}^{N} w_n^{(t)}\, \mathbf{1}[h_t(x_n) \neq y_n]}{\sum_{n=1}^{N} w_n^{(t)}}$$

  if $\epsilon_t \geq 1 - \frac{1}{K}$ **then** ▷ Weak learner learning condition violated
    Stop iteration (break)
  else if $\epsilon_t \leq 0$ **then** ▷ Perfect classification achieved
    Stop iteration (break)
  else
    Get base model h_t's vote

$$\alpha_t \;\leftarrow\; \ln\left(\frac{1 - \epsilon_t}{\epsilon_t}\right) + \ln(K - 1)$$

    Update weights:

$$w_n^{(t+1)} \;\leftarrow\; w_n^{(t)} \exp\Big(\alpha_t\, \mathbf{1}[h_t(x_n) \neq y_n]\Big)$$

    Normalize: $w_n^{(t+1)} \leftarrow \frac{w_n^{(t+1)}}{\sum_{m=1}^{N} w_m^{(t+1)}}$
  end if
end for
Output classifier:

$$H(x) \;=\; \arg\max_{k \in \{1,\ldots,K\}} \sum_{t=1}^{T} \alpha_t\, \mathbf{1}[h_t(x) = k]$$

A good starting point is to understand the differences between the two variants of AdaBoost. The version shown in the lecture slides is the **two-class** AdaBoost algorithm, while this assignment requires implementing **AdaBoost for multiclass classification (SAMME)**. The overall workflow is the same, but a few key components change to support $K > 2$ classes.

- **Number of classes**
  - Two-class AdaBoost assumes $y \in \{-1, +1\}$.
  - SAMME assumes $y \in \{1, \ldots, K\}$ for $K \geq 2$.
- **Weak learner output**
  - Two-class: $M_i(x) \in \{-1, +1\}$.
  - SAMME: $h_t(x) \in \{1, \ldots, K\}$ (a class label).
- **Weak learning condition (stopping rule)**
  - Two-class: stop if $\epsilon_i \geq 0.5$.

- SAMME: stop if $\epsilon_t \geq 1 - \frac{1}{K}$.

This reflects that random guessing in K classes has error $1 - \frac{1}{K}$.

- **Model vote (α) computation**
  - Two-class:
$$\alpha_i = \tfrac{1}{2} \ln\left(\frac{1-\epsilon_i}{\epsilon_i}\right)$$
  - SAMME:
$$\alpha_t = \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right) + \ln(K-1)$$

  The additional $\ln(K-1)$ term adjusts the vote strength for multiclass problems.

- **Weight update**
  - Both variants use the same idea:
$$w_n^{(t+1)} = w_n^{(t)} \exp\big(\alpha_t \, \mathbf{1}[h_t(x_n) \neq y_n]\big)$$

    Misclassified examples receive larger weights so that future learners focus on them.

- **Final prediction**
  - Two-class:
$$H(x) = \operatorname{sign}\left(\sum_i \alpha_i M_i(x)\right)$$
  - SAMME:
$$H(x) = \arg\max_{k \in \{1,\ldots,K\}} \sum_{t=1}^{T} \alpha_t \, \mathbf{1}[h_t(x) = k]$$

  Instead of a sign, SAMME selects the class with the largest total weighted vote.

2. (15 points) Answer in 2 to 3 sentences each.

   (a) Why does bagging primarily reduce variance, while boosting can reduce bias?

   (b) Why are decision stumps commonly used as base learners in boosting?

   (c) What happens to AdaBoost if the base learner performs worse than random guessing? Briefly explain why.

Part 2 Code

1. (35 points) The problem will focus on developing your own code for: K-`fold cross validation`.

   Your code will be evaluated using five standard classification models applied to a multi-class classification dataset.

   Dataset: We will be using the following dataset for the assignment.

   `Digits`: The `Digits` dataset comes prepackaged with `scikit-learn` (`sklearn.datasets.load_digits`). The dataset has 1797 points, 64 features, and 10 classes corresponding to ten numbers $0, 1, \ldots, 9$. The dataset was (likely) created from the following dataset:
   `https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits`

```
from sklearn.datasets import load_digits
digits = load_digits()
X, y = digits.data, digits.target
```

   Classification Methods. We will consider the following classification methods from `scikit-learn`:

   – Decision Tree Classifier: `DecisionTreeClassifier`,
   – Naive Bayes Classifier: `GaussianNB`,
   – Logistic Regression: `LogisticRegression`,

   together with the two ensembles, `RandomForestClassifier and AdaBoostClassifier` from Part 1. Use the following parameters for these methods (do not specify any additional parameters) when implementing `get_model(method, random_state)`:

   – `DecisionTreeClassifier: max_depth=10, random_state=random_state`
   – `GaussianNB:` (no parameters needed)
   – `LogisticRegression: penalty='l2', solver='lbfgs', max_iter=1000, random_state=random_state`
   – `RandomForestClassifier: tree_max_depth=BAGGING_TREE_MAX_DEPTH, num_estimators=BAGGING_NUM_ESTIMATORS,random_state=random_state`
   – `AdaBoostClassifier: num_estimators=BOOST_NUM_ESTIMATORS, random_state=random_state`

   You will have to develop **code** for the following three functions:

   (a) `get_model(method)`, which returns the appropriate classifier object with the parameters specified above, given the method specified as a string. The parameter `method` will be one of: `"DecisionTreeClassifier"`, `"GaussianNB"`, `"LogisticRegression"`, `"RandomForestClassifier"`, or `"AdaBoostClassifier"`.

   (b) `get_splits`(n, k, `seed`), which returns: randomized k 'almost equal' sized lists of unique disjoint indices from the set of all indices $\{0, \ldots, n-1\}$, where the randomization depends on the integer `seed`. By 'almost equal', we mean the cardinality of the lists can differ by at most 1. These k list of indices correspond to the k folds over which cross-validation will be performed. The function will have the following **output**:

i. a python list containing exactly k lists. Each of these sublists should be disjoint, each of size roughly $\frac{n}{k}$, contain elements from the set $\{0, 1, ..., n-1\}$ and must not contain repeated elements. The union of all the k sublists should include all elements in $\{0, \ldots, n-1\}$.

Input `seed` determines the randomization and the output should be the same every time we use the same `seed` for a given n, k, and should be (randomly) different for different values of the `seed`.

For example, `get_splits`(4, 2, 1) may return [[0,2], [1,3]], and the output must be same every time with the same input; `get_splits`(4, 2, 73) may return [[0,3], [1,2]]; `get_splits`(7, 2, 2) may return [[6,4,0,2], [1,5,3]]; `get_splits`(11, 3, 7) may return [[0,3,6,9], [1,4,7,10], [2,5,8]]. Within each fold, the indices may be in any order; any randomized, disjoint split is acceptable as long as it is deterministic for the given seed and fold sizes differ by at most 1.

For our tests, n would be less than 1200. `get_splits` would have a time limit of 10 seconds.

(c) `my_cross_val`(`method`, X, $\mathbf{y}$, `splits`), which runs k-fold cross-validation for `method` on the dataset $(X, \mathbf{y})$. The **input parameters** are:

i. `method`, which specifies the (class) name of one of the five classification methods under consideration,

ii. X, $\mathbf{y}$ which is the data for the classification problem

iii. `splits`: the output of the `get_splits method (len(splits) =` k`)`

The function will have the following **output**:

i. the test set error rates for each of the k folds.

The error should be measured as $\frac{\texttt{\# of wrong predictions}}{\texttt{\# of total predictions}}$. The (auto)grader will judge your solution as correct if the difference between the reported and the expected mean error rates is within 10^{-3}. `my_cross_val` would have a time limit of 2 minutes.

Within `my_cross_val`, strictly use the *splits* encoded in the input parameter `splits`. Do not define your splits for K-fold cross validation within this method.

Also, make sure that you are NOT inadvertently shuffling your training data during K-fold cross validation. The training examples (for any particular split) should be in the same order as the input X.

Use `my_cross_val` to return the error rates in each fold for k fold cross-validation for the specific `method` (one of `LinearSVC`, `SVC`, `LogisticRegression`, `RandomForestClassifier`, and `AdaBoostClassifier`) with the parameters outlined above.

2. (15 points) Data Challenges: Imbalance and Missing Values

   (a) **Class Imbalance (Report and Code Modification):** Imagine the dataset provided was highly imbalanced (e.g., 90% of samples are digit '1', and the remaining 10% are spread across digits '0, 2-9').

   - **Report Task:** Explain why accuracy (or error rate) is a poor metric for evaluating performance in this scenario. Then, define *precision*, *recall*, and explain how they combine to form the *F1-score*. Finally, explain why the **macro-average** F1-score is a suitable metric for imbalanced classification.
   - **Code Task:** Modify your `my_cross_val` function from Question 1 to calculate and return the macro-averaged F1-score instead of the error rate. You may need to import `f1_score` from `sklearn.metrics` for this. (No need to submit code for this task).

   (b) **Missing Values (Report Only):** Imagine 5% of the feature values in the dataset X were randomly replaced with missing values (`np.nan`). Before you could train a model, you would need to perform data imputation. Choose an imputation strategy (e.g., mean imputation, median imputation, or KNN imputation). In your report, justify why your chosen strategy would be appropriate for this dataset and discuss the potential drawbacks of an alternative method.

References

[1] J. Zhu, H. Zou, S. Rosset, and T. Hastie, "Multi-class AdaBoost," *Statistics and its Interface*, vol. 2, no. 3, pp. 349–360, 2009.